CSE 4137

Machine Learning
Module-05
Decision Tree



Md. Shahid Uz Zaman
Adjunct Faculty
Dept of CSE, BAUET
szaman22.ruet@gmail.com

Topics: DT

Week-08

- Introduction
- Implementation
- Python Example

DT(Decision Tree):SML

Week-08, c-3

What is DT?

•Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome.**

•In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node.** Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.

•The decisions or the test are performed on the basis of features of the given dataset.

•***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***

•It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.

•In order to build a tree, we use the **CART algorithm,** which stands for **Classification and Regression Tree algorithm.**

•A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.

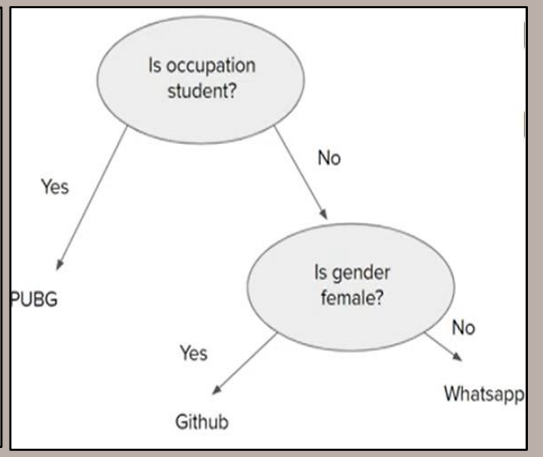
•Below diagram explains the general structure of a decision tree:

Example #1: DT

In the following dataset shows who downloads what content

Gender	Occupation	Suggestion
F	Student	PUBG
F	Programmer	Github
M	Programmer	Whatsapp
F	Programmer	Github
M	Student	PUBG
M	Student	PUBG

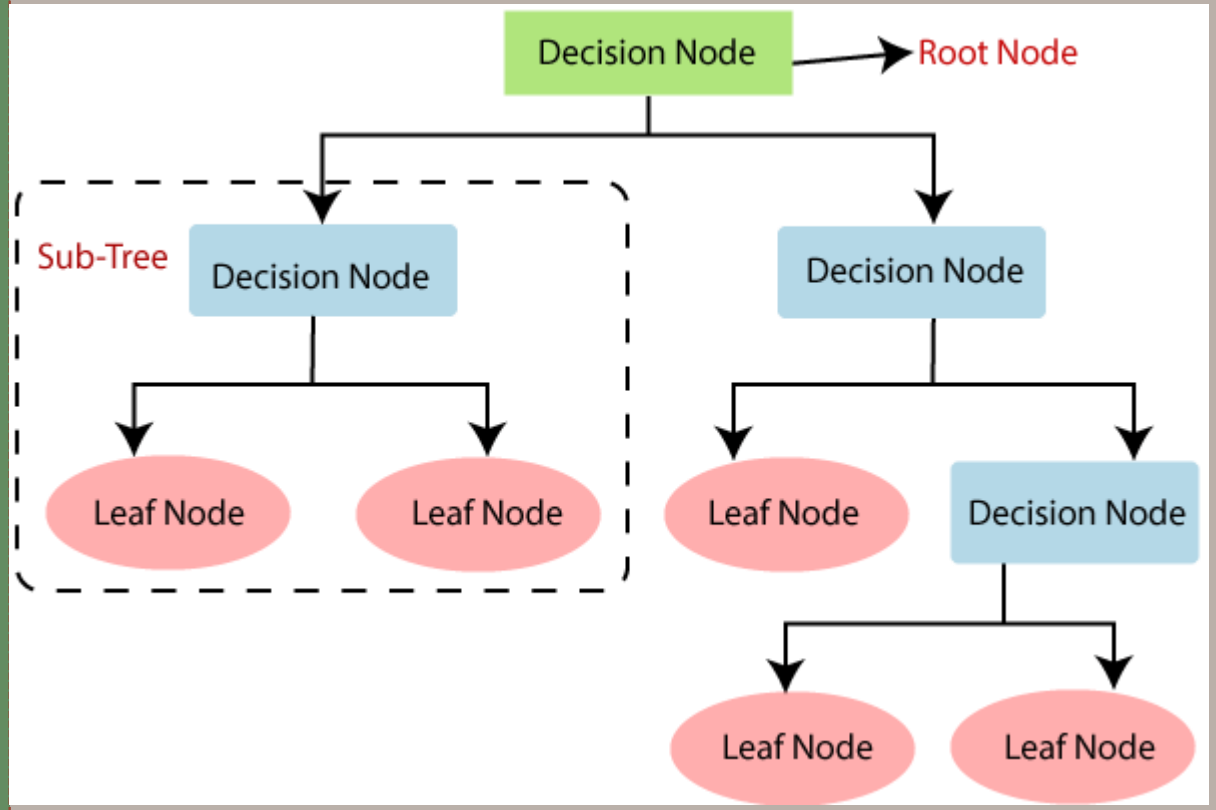
```
If occupation==student
    print(PUBG)
Else
    If gender==female
        print(Github)
    Else
        print(Whatsapp)
```



DT(Decision Tree):SML

Week-08, c-3

Components of DT



Working Steps of DT

- **Starting at the Root**: The algorithm begins at the top, called the “root node,” representing the entire dataset.
- **Asking the Best Questions:** It looks for the most important feature or question that splits the data into the most distinct groups. This is like asking a question at a fork in the tree.
- **Branching Out**: Based on the answer to that question, it divides the data into smaller subsets, creating new branches. Each branch represents a possible route through the tree.
- **Repeating the Process**: The algorithm continues asking questions and splitting the data at each branch until it reaches the final “leaf nodes,” representing the predicted outcomes or classifications.

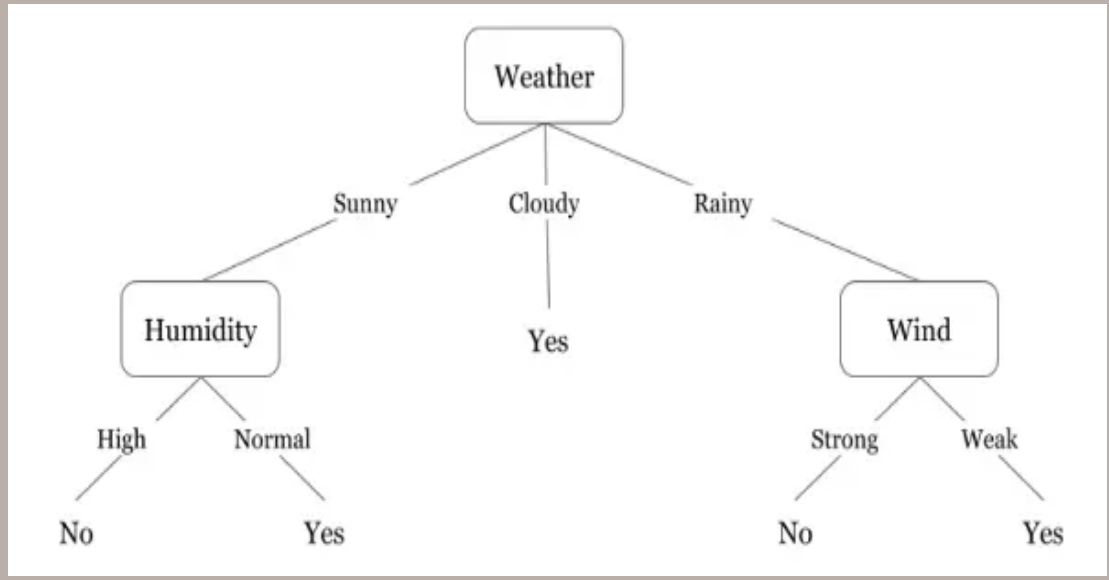
DT(Decision Tree):SML

Week-08, c-3

Example #2: DT

In the following dataset shows who plays tennis at different weather conditions

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No



In the diagram the tree will first ask what is the weather? Is it sunny, cloudy, or rainy? If yes then it will go to the next feature which is humidity and wind. It will again check if there is a strong wind or weak, if it's a weak wind and it's rainy then the person may go and play.

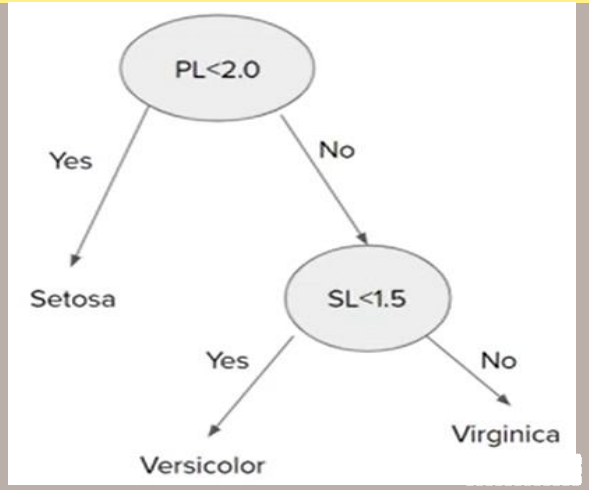
DT(Decision Tree):SML

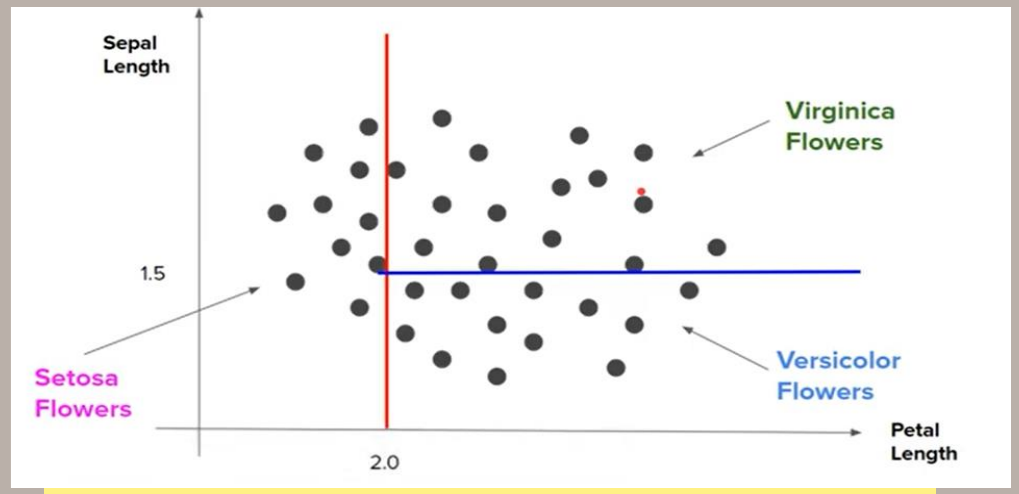
Week-08, c-3

Example #3: DT

In the following dataset shows the type of flower depends on petal and sepal length

Petal Length	Sepal Length	Type
1.34	0.34	Setosa
3.45	1.45	Versicolor
1.69	0.98	Setosa
2.56	1.79	Virginica
3.00	1.13	Versicolor
1.3	0.88	Setosa





DT basically cuts the coordinates

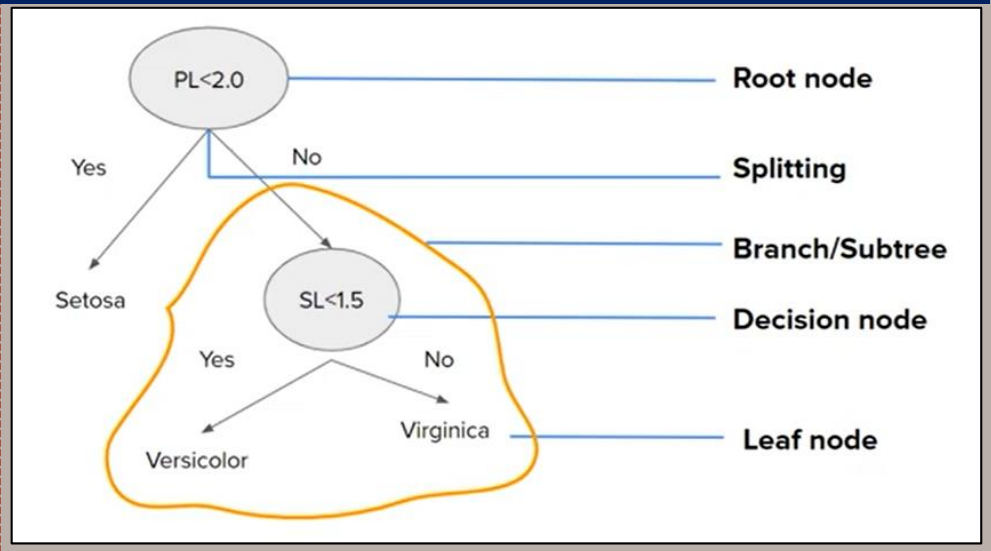
Pseducode: DT

- Begin with your training dataset, which should have some feature variables and classification or regression output.
- Determine the “best feature” in the dataset to split the data on; more on how we define “best feature” later
- Split the data into subsets that contain the correct values for this best feature. This splitting basically defines a node on the tree i.e each node is a splitting point based on a certain feature from our data.
- Recursively generate new tree nodes by using the subset of data created from step 3.

DT(Decision Tree):SML

Week-08, c-3

Terminology: DT



Important Questions: DT

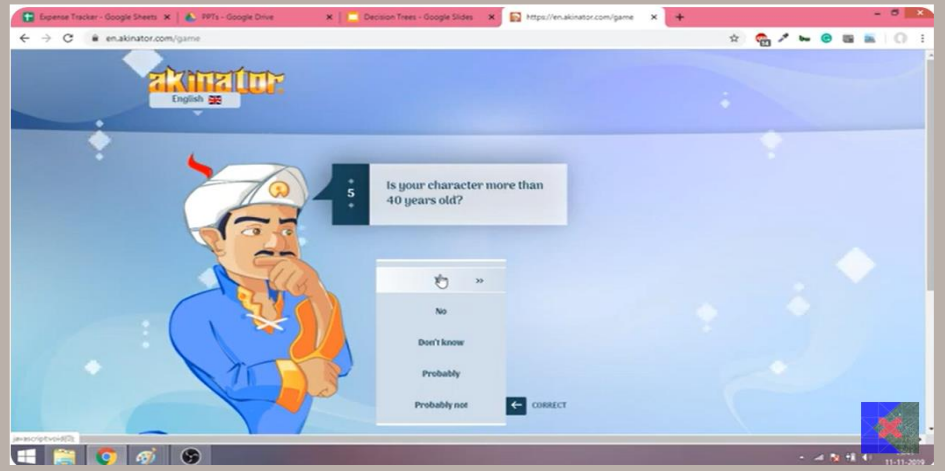
How to decide which column should be considered as root node?

How to select subsequent decision nodes?

How to decide splitting criteria in case of numerical columns?

Advantages	Disadvantages
Intuitive and easy to understand	Overfitting
Minimal data preparation is required	Prone to errors for imbalanced datasets
The cost of using the tree for inference is **logarithmic** in the number of data points used to train the tree	

Puzzle Game: DT



Topics: DT

Week-08

How does DT works?

DT(Decision Tree):SML

Week-08, c-3

Entropy

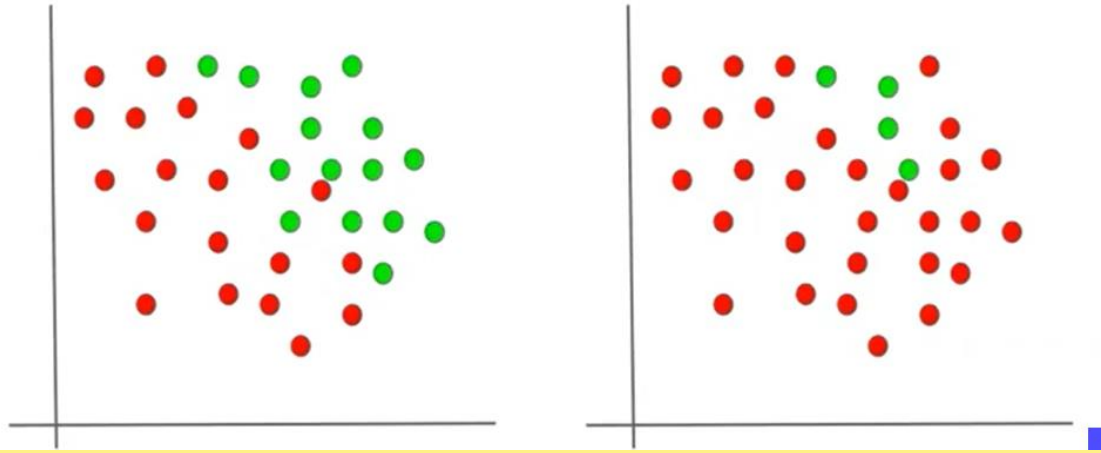
In the most layman terms, Entropy is nothing but the measure of disorder. Or you can also call it the measure of purity/impurity. Let's see an example...



In the above example, molecules are most compact in ice and less compact in vapor that means ice has less disorders but vapor has most disorders. So vapor has highest entropy than water and ice.

Example: Entropy

In the following yes/no dataset. In which dataset we can predict easily?



More knowledge less entropy

The first dataset has higher entropy as here more difficult to predict than 2nd one.

Entropy is the knowledge of uncertinity.

DT(Decision Tree):SML

Week-08, c-3

Calculate Entropy

The mathematical formula for entropy is:

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

Where 'Pi' is simply the frequentist probability of an element/class 'i' in our data.

For e.g if our data has only 2 class labels Yes and No.

$$E(D) = -p_{yes}\log_2(p_{yes}) - p_{no}\log_2(p_{no})$$

Salary	Age	Purchase
20000	21	Yes
10000	45	No
60000	27	Yes
15000	31	No
12000	18	No

$H(d) = -P_y \log_2(P_y) - P_n \log_2(P_n)$

$H(d) = -2/5\log_2(2/5) - 3/5\log_2(3/5)$

$H(d) = 0.97$

Salary	Age	Purchase
34000	31	No
15000	25	No
69000	57	Yes
25000	21	No
32000	28	No

$H(d) = -P_y \log_2(P_y) - P_n \log_2(P_n)$

$H(d) = -1/5\log_2(1/5) - 4/5\log_2(4/5)$

$H(d) = 0.72$

In the 2nd second dataset it is easy to predict so less entropy and 1st dataset is difficult to predict so higher entropy, we can calculate the entropy. The calculation shows that 1st dataset (0.97) has higher entropy than 2nd one (0.72)

Calculate Entropy

Salary	Age	Purchase
20000	21	No
10000	45	No
60000	27	No
15000	31	No
12000	18	No

$H(d) = -P_y \log_2(P_y) - P_n \log_2(P_n)$

$H(d) = -0/5\log_2(0/5) - 5/5\log_2(5/5)$

$H(d) = 0$

DT(Decision Tree):SML

Week-08, c-3

Calculate Entropy: 3 class

Salary	Age	Purchase
20000	21	Yes
10000	45	No
60000	27	Yes
15000	31	No
30000	30	Maybe
12000	18	No
40000	40	Maybe
20000	20	Maybe

$$H(d) = - P_y \log_2(P_y) - P_n \log_2(P_n) - P_m \log_2(P_m)$$

$$H(d) = -2/8 \log_2(2/8) - 3/8 \log_2(3/8) - 3/8 \log_2(3/8)$$

$$H(d) = 1.56$$

Observations: 'Entropy

- More the uncertainty more is entropy
- For a 2 class problem the min entropy is 0 and the max is 1
- For more than 2 classes the min entropy is 0 but the max can be greater than 1
- Both $\log_2$ or $\log_e$ can be used to calculate entropy

DT(Decision Tree):SML

Week-08, c-3

Information Gain

Information Gain, is a metric used to train Decision Trees. Specifically, this metric measures the quality of a split.

The information gain is based on the decrease in entropy after a data-set is split on an attribute.Constructing a decision tree is all about finding attribute that returns the highest information gain

Information Gain = E(Parent) - {Weighted Average} * E{Children}

Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

Step-1: Calculate E of Root

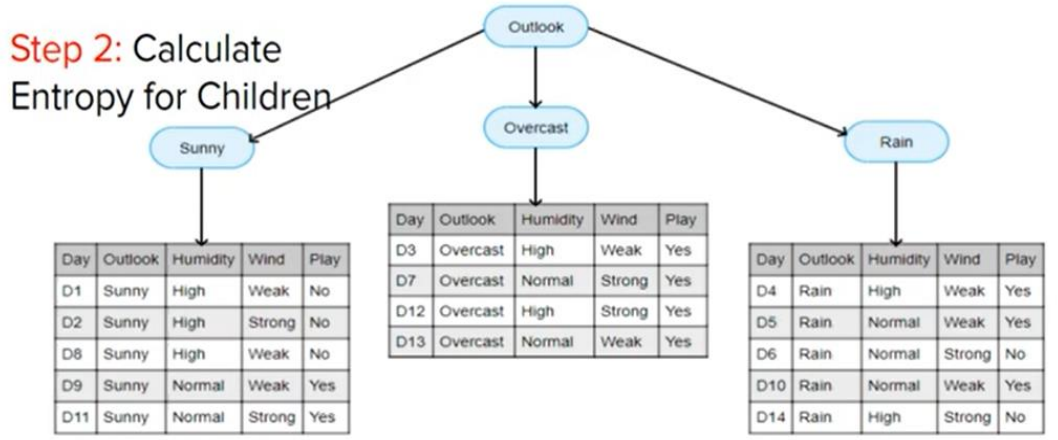
$$E(P) = -p_y \log_2(p_y) - p_n \log_2(p_n)$$

$$= 9/14 \log_2(9/14) - 5/14 \log_2(5/14)$$

$$E(P) = \mathbf{0.94}$$

Step-2: After first split

Step 2: Calculate Entropy for Children



Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes

Day	Outlook	Humidity	Wind	Play
D3	Overcast	High	Weak	Yes
D7	Overcast	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes

Day	Outlook	Humidity	Wind	Play
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D10	Rain	Normal	Weak	Yes
D14	Rain	High	Strong	No

E(S) =	E(O) =	E(R) =
-2/5log(2/5) - 3/5log(3/5)	-5/5log(5/5) - 0/5log(0/5)	-3/5log(3/5) - 2/5log(2/5)
E(S)= 0.97	E(O)= 0	E(S)= 0.97

DT(Decision Tree):SML

Week-08, c-3

Step-3: Weighted Entropy

Step 3 : Calculate weighted Entropy of Children

Weighted Entropy = 5/14 * 0.97 + 4/14 * 0 + 5/14 * 0.97

W.E(Children) = **0.69**

P(Overcast) is a leaf node as it's entropy is 0

For 0 entropy, no need to split

Step-4: Information Gain

Step 4 : Calculate Information Gain

Information Gain = E(Parent) - {Weighted Average} * E(Children)

IG = **0.97 - 0.69 = 0.28**

So the information gain(or the decrease in entropy/impurity) when you split this data on the basis of **Outlook** condition/column is **0.28**

Step-5: Col Information Gain

Step 5 : Calculate Information Gain for all the columns

Whichever column has the highest Information Gain(maximum decrease in entropy) the algorithm will select that column to split the data.

Step-6: All Information Gain

Step 6 : Find Information Gain recursively

Decision tree then applies a recursive greedy search algorithm in top bottom fashion to find Information Gain at every level of the tree.

Once a leaf node is reached (Entropy = 0), no more splitting is done.

Topics: DT

Week-08

Python Example

DT (Python Example)

Week-08, c-3

Example: Saray Dataset

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import train_test_split
from sklearn import tree
```

```
df = pd.read_csv("f:\\python\\salary.csv")
df.head()
```

```
X = df.drop('Salar_More_1Lac',axis='columns')
y = df['Salar_More_1Lac']
```

```
le_Gender = LabelEncoder()
le_Education_Level = LabelEncoder()
le_Job_Title = LabelEncoder()
```

```
X['Gender_n'] = le_Gender.fit_transform(X['Gender'])
X['Education_Level_n'] =
le_Education_Level.fit_transform(X['Education Level'])
X['Job_Title_n'] = le_Job_Title.fit_transform(X['Job Title'])
```

```
X.drop(['Gender','Education Level','Job
Title'],axis='columns',inplace=True)
```

```
X.isnull().sum()
```

```
X['Age'].fillna(value=df['Age'].mean() , inplace=True)
X['Years of Experience'].fillna(valu`e=df['Years of Experience'].mean()
, inplace=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2)
model = tree.DecisionTreeClassifier()
model.fit(X_train, y_train)
```

```
y_pred=model.predict(X_test)
cm=confusion_matrix(y_test,y_pred)
```

```
array([[550,  16],
       [ 10, 765]], dtype=int64)
```

```
print(classification_report(y_test,y_pred))
```

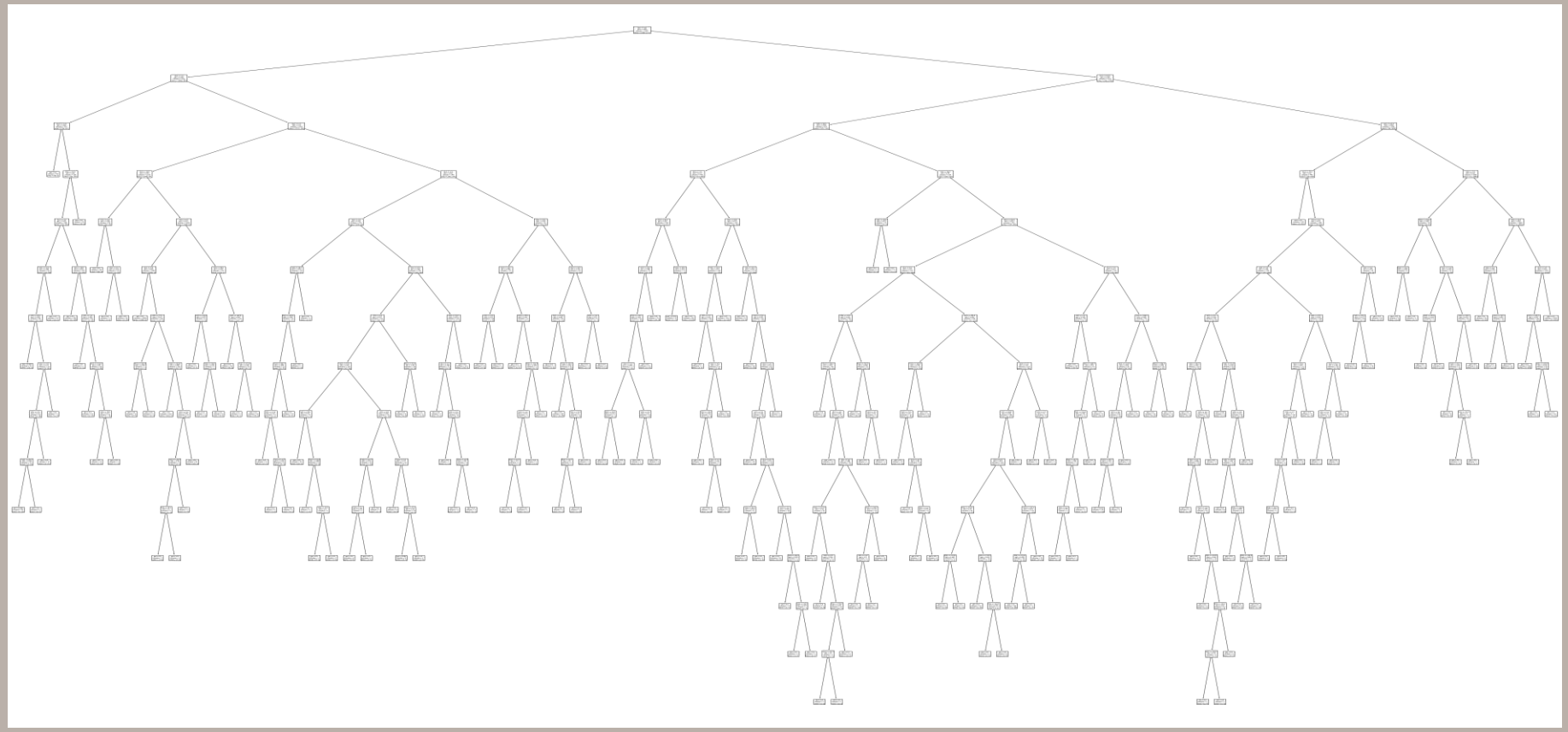
	precision	recall	f1-score	support
0	0.98	0.97	0.98	566
1	0.98	0.99	0.98	775
accuracy			0.98	1341

DT (Python Example)

Week-08, c-3

Example: Saray Dataset

```
from sklearn.tree import plot_tree
from matplotlib.pylab import rcParams
rcParams['figure.figsize']=80,50
plot_tree(model)
```



DT (Python Example)

Week-08, c-3

Example: Iris Dataset

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, classification_report
```

```
iris=load_iris()
X=pd.DataFrame(iris.data,columns=iris.feature_names)
y=iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
```

```
dt=DecisionTreeClassifier()
dt.fit(X_train,y_train)
```

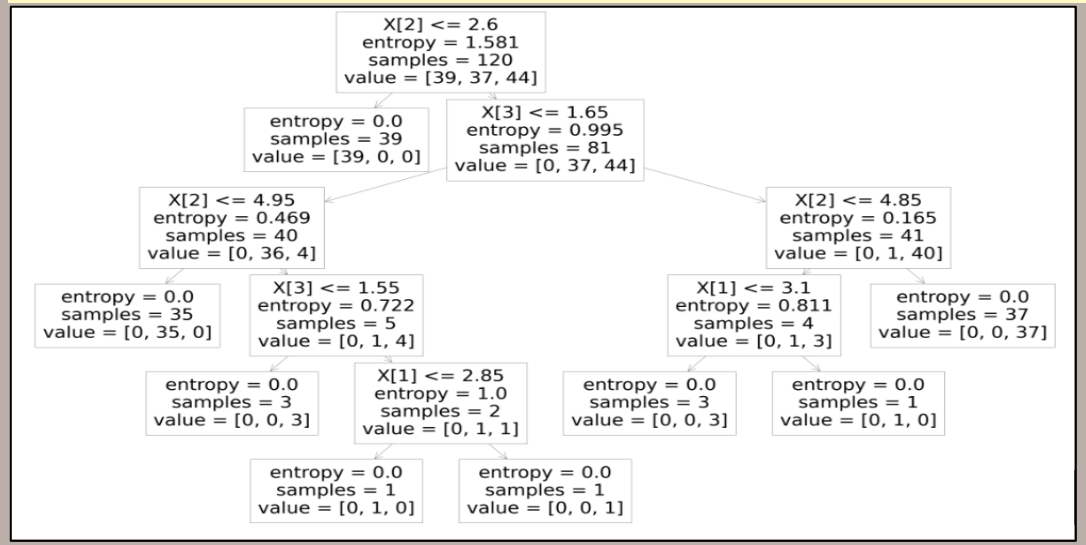
```
y_pred=dt.predict(X_test)
cm=confusion_matrix(y_test,y_pred)
cm
```

```
array([[11,  0,  0],
       [ 0, 12,  1],
       [ 0,  0,  6]], dtype=int64)
```

```
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	1.00	0.92	0.96	13
2	0.86	1.00	0.92	6
accuracy			0.97	30

```
from sklearn.tree import plot_tree
from matplotlib.pylab import rcParams
rcParams['figure.figsize']=80,50
plot_tree(dt)
```



Week-08

Overfitting & Underfitting

Overfitting & Underfitting

Week-08, c-3

Overfitting

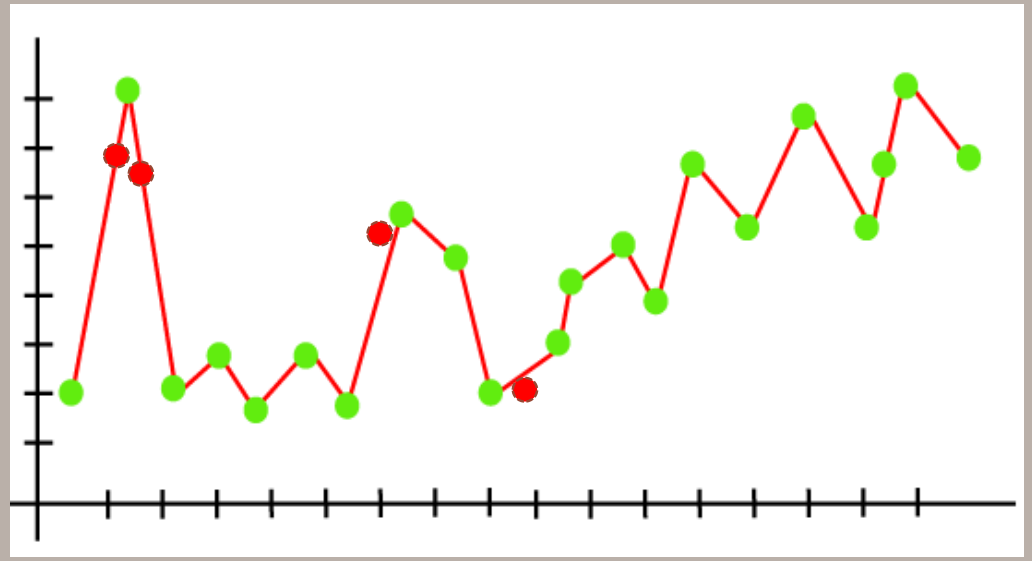
Overfitting occurs when our machine learning model tries to cover all the data points or more than the required data points present in the given dataset. Because of this, the model starts caching noise and inaccurate values present in the dataset, and all these factors reduce the efficiency and accuracy of the model. The overfitted model has low bias and high variance.

The chances of occurrence of overfitting increase as much we provide training to our model. It means the more we train our model, the more chances of occurring the overfitted model.

Overfitting is the main problem that occurs in supervised learning.

Exmaple: Overfitting

The concept of the overfitting can be understood by the below graph of the linear regression output:
Suppose here x-axis and y-axis denotes number of hours of study and GPA obtained by students



The model works very well for training but not well for testing. Ex: look at the prediction for red point, they are not good predictions.
As we can see from the above graph, the model tries to cover all the data points present in the scatter plot. It may look efficient, but in reality, it is not so. Because the goal of the regression model to find the best fit line, but here we have not got any best fit, so, it will generate the prediction errors.

Overfitting & Underfitting

Week-08, c-3

How to avoid the Overfitting

Both overfitting and underfitting cause the degraded performance of the machine learning model. But the main cause is overfitting, so there are some ways by which we can reduce the occurrence of overfitting in our model.

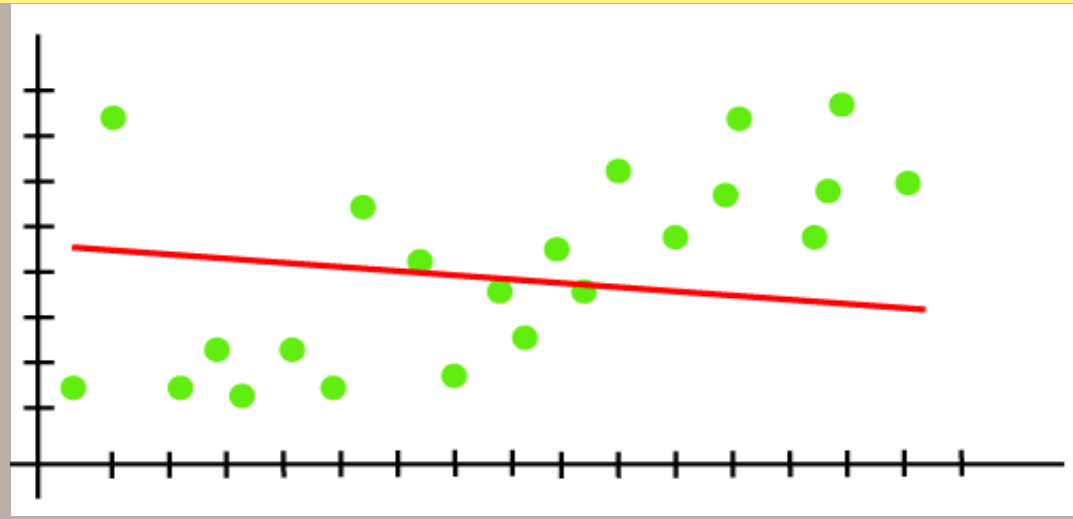
- Cross-Validation
- Training with more data
- Removing features
- Early stopping the training
- Regularization
- Ensembling

Underfitting

Underfitting occurs when our machine learning model is not able to capture the underlying trend of the data. To avoid the overfitting in the model, the fed of training data can be stopped at an early stage, due to which the model may not learn enough from the training data. As a result, it may fail to find the best fit of the dominant trend in the data.
In the case of underfitting, the model is not able to learn enough from the training data, and hence it reduces the accuracy and produces unreliable predictions.
An underfitted model has high bias and low variance.

Example: Underfitting

We can understand the underfitting using below output of the linear regression model:



As we can see from the above diagram, the model is unable to capture the data points present in the plot.

How to avoid underfitting:

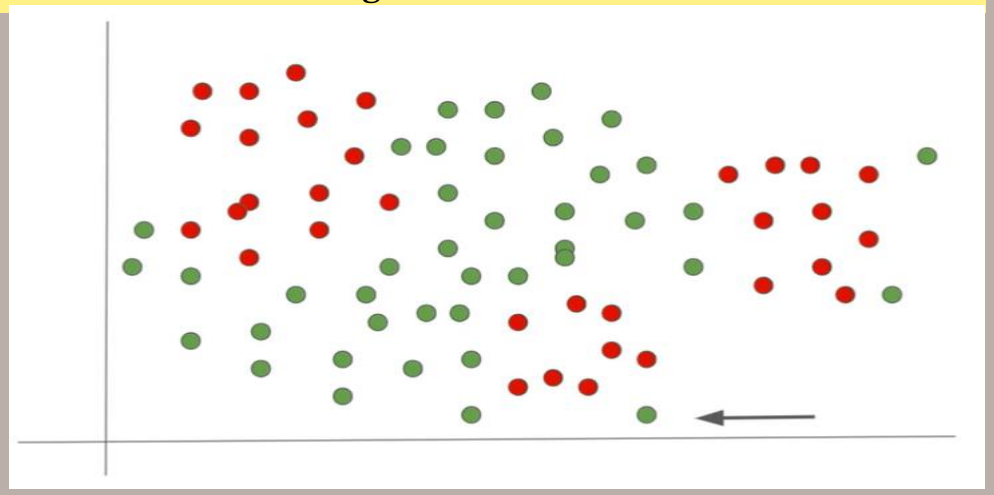
- By increasing the training time of the model.
- By increasing the number of features.

Overfitting & Underfitting

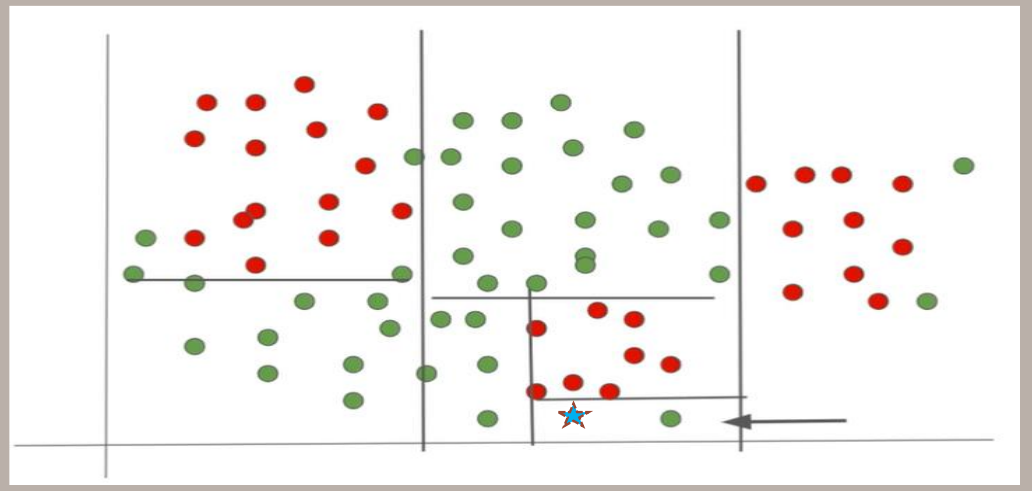
Week-08, c-3

Overfitting in DT

Consider the following dataset



Example: After few split



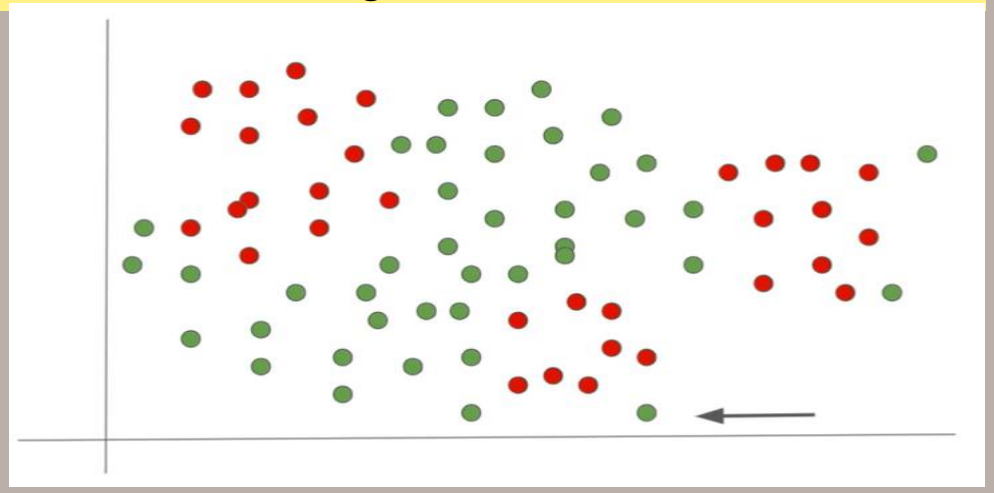
Consider the point pointed by an arrow is only one point in the box, so all points inside the box classified as green level although many points are very near to red level like blue '*' point. Although blue '*' should classified as red level but due to overfitting it will be classified as green level.

Overfitting & Underfitting

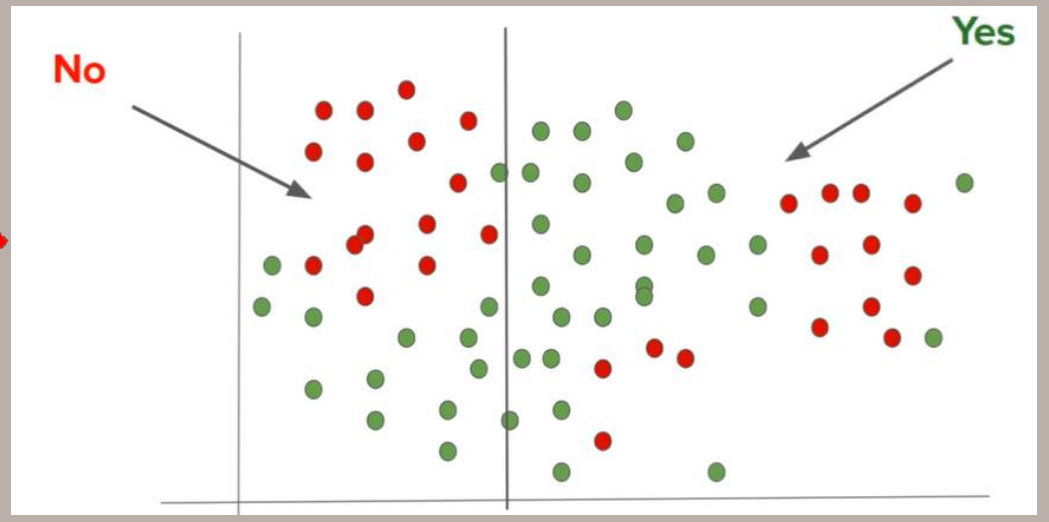
Week-08, c-3

Underfitting in DT

Consider the following dataset



Example: one split

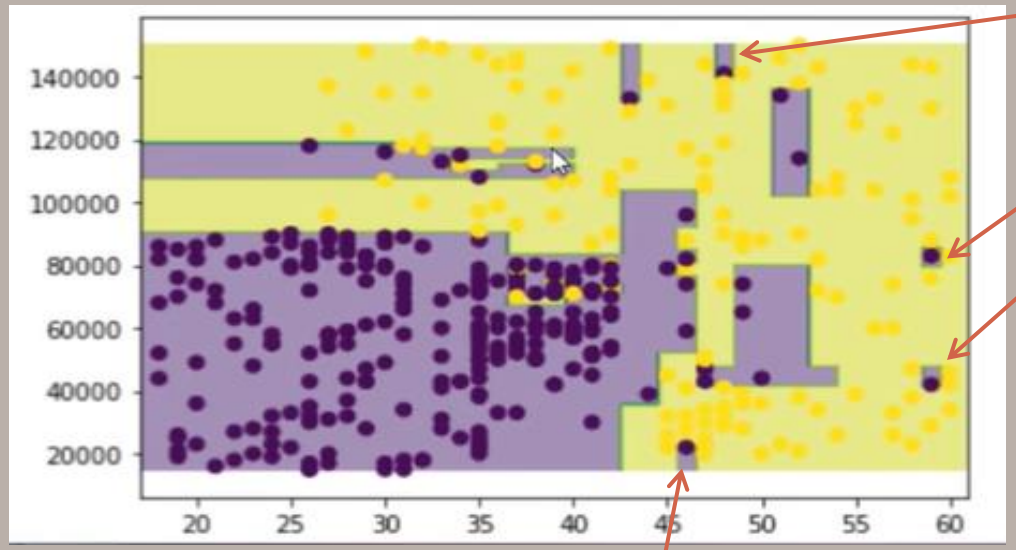


Here Depth=1 (only one split)
So left side is predicted as No and right side predicted as Yes but it is somewhat incorrect.

Overfitting & Underfitting

Week-08, c-3

Example: Overfitting in DT

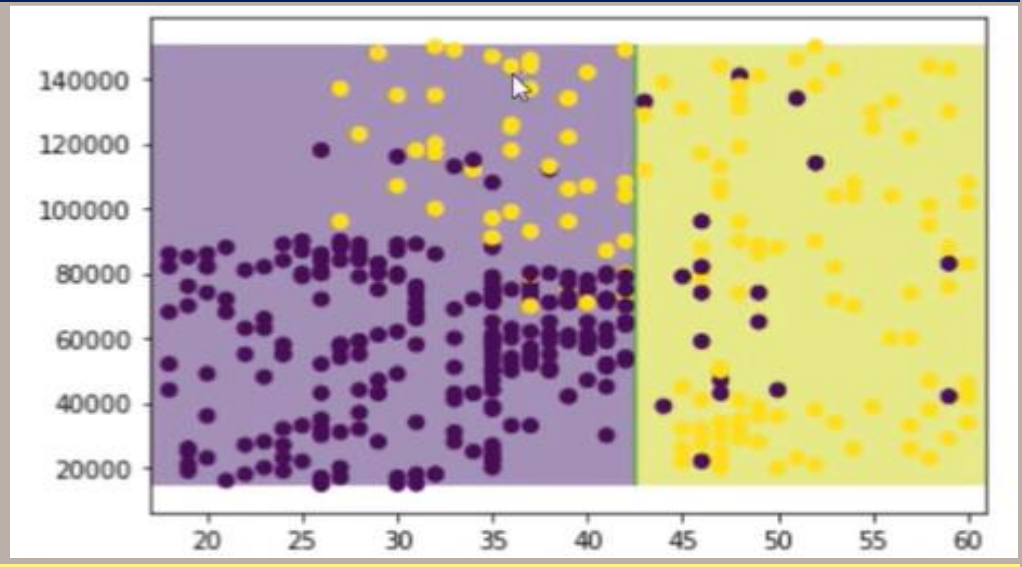


This point should be in yellow class

This point should be in yellow class

To minimize overfitting & underfittin
We need to optimize the depth of split

Example: Underfitting in DT



There are so many yellow points in left side and vice -versa

Overfitting & Underfitting

Week-08, c-3

Python Example

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report
from sklearn.tree import DecisionTreeClassifier
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df=pd.read_csv("F:\\python\\Juicequality.csv")
df.isnull().sum()
```

```
df.quality.value_counts()
```

```
quality_mapping={3:0,4:1,5:2,6:3,7:4,8:5}
df.loc[:,"quality"]=df.quality.map(quality_mapping)
df.quality.value_counts()
```

```
X=df.drop("quality",axis='columns')
y=df.quality
X_train,X_test,y_train,y_test=train_test_split(X,y,test_
size=0.2)
```

```
dt=DecisionTreeClassifier(max_depth=1)
dt.fit(X_train,y_train)
```

```
y_pred_test=dt.predict(X_test)
y_pred_train=dt.predict(X_train)
```

```
print(accuracy_score(y_train,y_pred_train))
```

```
0.512118428459734
```

```
print(accuracy_score(y_test,y_pred_test))
```

```
0.503125
```

```
from sklearn import tree
%matplotlib inline
train_accuracies=[]
test_accuracies=[]
for depth in range(1,25):
    tree_model=DecisionTreeClassifier(max_depth=depth)
    tree_model.fit(X_train,y_train)
    train_pred=tree_model.predict(X_train)
    test_pred=tree_model.predict(X_test)
    train_accuracy=accuracy_score(y_train,train_pred)
    test_accuracy=accuracy_score(y_test,test_pred)

    train_accuracies.append(train_accuracy)
    test_accuracies.append(test_accuracy)
```

Overfitting & Underfitting

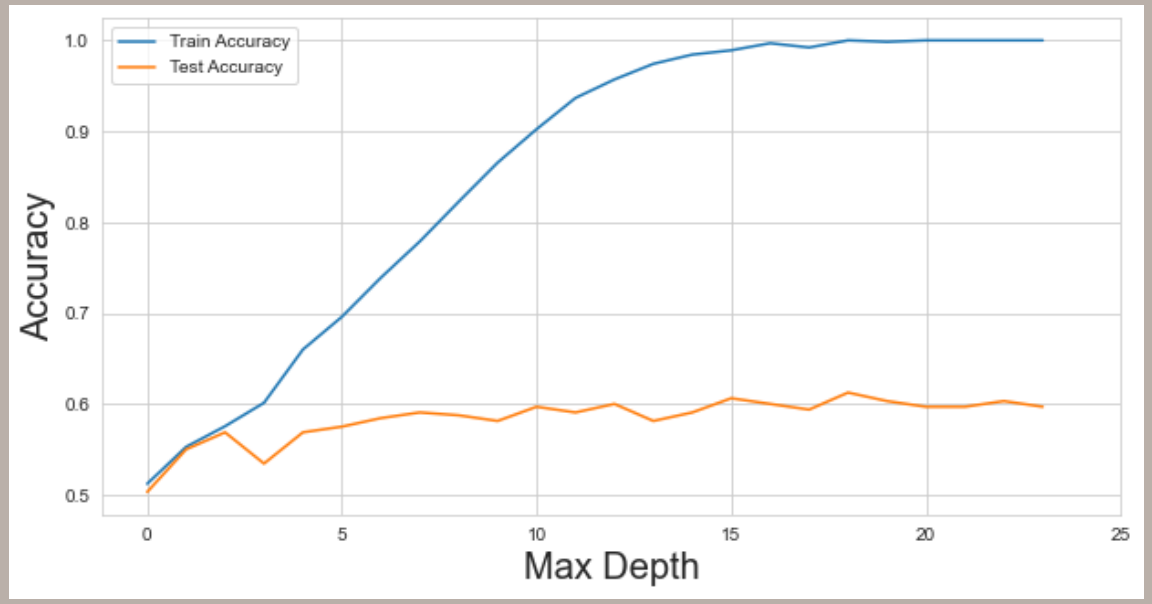
Week-08, c-3

Python Example

```
from sklearn import tree
%matplotlib inline
train_accuracies=[]
test_accuracies=[]
for depth in range(1,25):
    tree_model=DecisionTreeClassifier(max_depth=depth)
    tree_model.fit(X_train,y_train)
    train_pred=tree_model.predict(X_train)
    test_pred=tree_model.predict(X_test)
    train_accuracy=accuracy_score(y_train,train_pred)
    test_accuracy=accuracy_score(y_test,test_pred)

    train_accuracies.append(train_accuracy)
    test_accuracies.append(test_accuracy)
```

```
plt.figure(figsize=(10,5))
sns.set_style("whitegrid")
plt.plot(train_accuracies,label="Train Accuracy")
plt.plot(test_accuracies,label="Test Accuracy")
plt.xticks(range(0,26,5))
plt.legend(loc="upper left")
plt.xlabel("Max Depth",size=20)
plt.ylabel("Accuracy",size=20)
```



We can see clearly that train accuracy is high but test accuracy is low. It is overfitting. The training accuracy is increasing with the depth of the tree.

End of Chapter